

# Hyperlink Spoofing and the Modern Web

**Hyperlink Spoofing and the Modern Web** - Author not stated, [blogs.msdn.com](https://blogs.msdn.com).

- Published: date not stated
- Original: <[https://web.archive.org/web/20120526051852/http://blogs.msdn.com/cfs-file.ashx/\\_key/communityserver-blogs-components-weblogfiles/00-00-00-47-30/5340.Hyperlink-Spoofing-and-the-Modern-Web\\_2800\\_final\\_2900.pdf](https://web.archive.org/web/20120526051852/http://blogs.msdn.com/cfs-file.ashx/_key/communityserver-blogs-components-weblogfiles/00-00-00-47-30/5340.Hyperlink-Spoofing-and-the-Modern-Web_2800_final_2900.pdf)>
- Preserved from: [https://web.archive.org/web/20120526051852/http://blogs.msdn.com/cfs-file.ashx/\\_key/communityserver-blogs-components-weblogfiles/00-00-00-47-30/5340.Hyperlink-Spoofing-and-the-Modern-Web\\_2800\\_final\\_2900.pdf](https://web.archive.org/web/20120526051852/http://blogs.msdn.com/cfs-file.ashx/_key/communityserver-blogs-components-weblogfiles/00-00-00-47-30/5340.Hyperlink-Spoofing-and-the-Modern-Web_2800_final_2900.pdf) (manual-import) on 2026-08-07
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

## Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

--- page 1 ---

Hyperlink Spoofing and the Modern Web The status quo and thoughts on how to move forward  
David Ross MSRC Engineering Team, Microsoft 4/25/2012 Table of Contents Introduction  
..... 1 Hyperlink  
Basics ..... 2 An  
Increasing Problem ..... 3  
Harder than it Looks ..... 5  
What do other Sites / Apps do? ..... 6  
Platform Example #1 ..... 6  
Platform Example #2 ..... 7  
Platform Example #3 ..... 9  
Platform Example #4 ..... 10  
Platform Example #5 ..... 11  
Hyperlink Spoofing Test Result Summary .....  
13 McAfee .....  
13 Improving the Status Quo .....  
15 Too Many Hyperlinks .....  
19 Conclusion / Recommendations  
..... 19 Acknowledgements  
..... 20 Introduction  
Hyperlinks in web content are a fundamental technology underpinning the World Wide Web.  
Presented with a hyperlink, a user will make a decision to click based on the hyperlink itself and

the surrounding environment online safety by facilitating client-side malware, invasion of privacy, and services they trust (eg: ClickJacking). Unfortunately, users today are not empowered with consistent mechanisms intended to enable informed decision making about hyperlinks. New trends such as short-links and the rise of social networking are only making this situation more problematic.

--- page 2 ---

While hyperlink spoofing as a social engineering threat or a PEBKAC issue, there is a good argument that existing hyperlink related design practices are inconsistent and suboptimal with regard to security. If only for those reasons, the hyperlink spoofing threat is worthy of some thought. This paper enumerates the problem scenarios, identifies where issues exist on popular social networking sites / apps, and posits practical, security-optimized solutions. Hyperlink Basics Browsers and other software which present hyperlinks to the user will often provide some indication of where a particular hyperlink might navigate the browser. For example, in Internet Explorer, hovering over a hyperlink will show a visual indication of its target URI at the bottom of the browser window. (Fig. A) Fig. A Since the inception of the web, hyperlinks themselves have not been considered trustworthy. The text for any given hyperlink may be a particular URI, however the actual target of navigation in the browser may be a completely different URI. Moreover, clicking a hyperlink may immediately result in the browser navigating elsewhere, facilitated by a server-side redirect, META Refresh, browser plugins, or client-side script. A common question from users How do I know where this hyperlink is going to take The answer in the browser security space There is a cross-browser security guarantee enforcing the trustworthiness of UX such as the address bar and lock icon, however this only allows users to make a determination about the trustworthiness of a page once navigation is complete. For some scenarios (eg: hyperlinks leading to offensive images or malware) this is clearly lacking.

--- page 3 ---

An Increasing Problem The untrustworthy nature of hyperlinks is proving to be increasingly less tolerable on the modern web. Consider the following Tweet (Fig. B): Fig. B (@homerentaldeals is shown above strictly as a representative example no wrongdoing is claimed.) Elsewhere on the web, savvy users can make a very rudimentary trust decision about whether to click on a given hyperlink by taking into account the trustworthiness of its surrounding page and the perceived hyperlink destination presented in the browser UX. If the source page URI itself is trustworthy, users can trust that the script content within the page will not redirect their navigation to another site. For modern social networking sites and apps (eg: Twitter), this level of trust is a given. Users can thus hover over any hyperlink on the page to make a trust decision about the URI displayed in the browser UX. Even if the hyperlink text is a spoof (eg: a hyperlink in a forum post) they still in theory can trust in the browser UX not to lie. Specifically, it is possible to determine that a hyperlink on a trustworthy page that the browser UX indicates will navigate to <http://www.microsoft.com> will actually navigate to <http://www.microsoft.com>. The paradigm described above is starting to break, and Twitter presents a great example of how. Though the Twitter web site is trusted, the contents of individual tweets are not, by design. Twitter has introduced its own short-t.co undoubtedly delivers on that goal, as URIs pointing to malware, XSS, and other attacks may be filtered out. However, <http://t.co> as the destination of a hyperlink is completely ambiguous. This arguably s by no means the only popular short-link service. Given the

Harder than it Looks By changing the target of the redirect after the initial lookup occurs, it was possible to easily construct a Tweet that would spoof the hyperlink target. (Fig. D) Fig. D The hyperlink in the tweet would actually navigate the user to <http://evil.com>. To be clear, this spoofing

scenario is no longer possible as Twitter has disabled full hyperlink resolution. Upon clicking the hyperlink, the browser would navigate to the t.co URI, which subsequently triggered navigation to the bit.ly URI. The bit.ly URI in turn navigated to the malicious redirector, which was then updated to point to evil.com, though it originally referenced the benign Microsoft.com URI. This hyperlink spoofing scenario is a good example of a TOCTOU problem. But beyond TOCTOU, a malicious redirector could also simply source IP or other characteristics of the request. The Twitter hyperlink resolution feature could have been described as a usability helper rather than a real security feature. That is, its intention was to allow users to more easily determine if they want to click on hyperlinks that are not intentionally spoofing their destination. But if some small number of hyperlinks do actually spoof their destination, people might start to lose faith in this type of feature to provide useful results even in the non-malicious use case. The sole purpose of the Twitter hyperlink resolution feature was to identify the destination of hyperlinks. That through this feature it was possible to spoof the destination of a hyperlink is actually a good argument that the original implementation of the feature made hyperlinks less trustworthy in exchange for convenience. Previously, users could be assured that if they hover over a hyperlink on a trustworthy page, and the browser UX reports the URI as <http://www.microsoft.com>, clicking on the hyperlink will result in a navigation through <http://www.microsoft.com>. Microsoft.com could certainly redirect elsewhere, but <http://www.microsoft.com> is less likely to be malicious in and of itself. But with the Twitter hyperlink resolution feature any hyperlink in Twitter could effectively spoof - Twitter UX might have reported a hyperlink as <http://www.microsoft.com>, the hyperlink itself could

--- page 7 ---

navigate anywhere. Ignoring browser vulnerabilities, implementation bugs, reflected cross-site scripting, unchecked redirects, and homograph attacks, this sort of spoofing should not be possible. Twitter mitigated the problem by disabling full hyperlink resolution. Unfortunately, in doing this they abandoned an attempt to provide better information to users about the true destination of hyperlinks. So is full hyperlink resolution impossible to achieve without introducing a spoofing vulnerability? This paper makes the case that it is possible to build a secure hyperlink resolution feature without compromising on security / privacy in other respects. What do other Sites / Apps do? As of now there is no standard mechanism for handling hyperlinks in social media updates. Different behavior has been observed across various popular social networking platforms. Note: Content below is modified / redacted to discourage direct comparison between platforms. It was not feasible to obscure the identity of Twitter or mcaf.ee. Platform Example #1 Fig. E In Platform Example #1, two hyperlinks with independent targets are presented when a hyperlink is shared in a status update. (Fig. E) While the second URI is authentic and triggers navigation to the specified location, the first URI may appear as a bogus destination and navigate unconditionally to a short-link. In the example above, the top hyperlink says <http://www.microsoft.com> but will ultimately navigate to <http://evil.com>. The browser shows the short-link on hover. Generally speaking, anchors on the web may be a surprise for the web platform as a whole. The behavior of Platform Example #1 has been noted in the past and addressed to a limited extent by a partnership with a third party URL reputation service. (This service allows Platform Example #1 to identify and eliminate known-bad hyperlinks.) I still reasonable to argue that Platform Example #1 is in a position to better indicate the ultimate destination of all URIs in status updates.

Platform Example #2 Fig. F A status update posted on Platform Example #2 actually removes the short-link entirely and replaces it with a custom redirector hosted by Platform Example #2. (Fig. F) From a security perspective, this removes the potential for a TOCTOU issue, as seen in the case of Twitter and Platform Example #1. A more convincing spoof scenario on Platform Example #2 would be to submit a hyperlink targeting a malicious destination which has been configured so as to display safe hyperlink text, for example <http://www.microsoft.com>. (Fig. F-2) Fig. F-2 -2 is the domain of the Platform Example #2 web site. Clicking the <http://www.microsoft.com/> hyperlink will navigate the user to a page on the Platform Example #2 -3). This page frames the malicious destination site (not <http://www.microsoft.com>). It could be argued that users already understand hyperlink text may be spoofed. Still, in this case the hyperlink is displayed within the context of the trusted Platform Example #2 site.

Fig. F-3 The top level Platform Example #2 page displays the malicious page URI (Fig. F-3). The malicious page can immediately navigate the top level browser to a different location, though this would not enable spoofing of content perceived to be hosted on Platform Example #2 itself. (Notice the malicious page URI displayed clearly in the top-level page.) Hyperlinks in tweets that propagate to Platform Example #2 appear as t.co hyperlinks in the browser, though they are also wrapped by a custom Platform Example #2 redirector. This shown by the browser as the hyperlink destination. Clicking on a hyperlink from Twitter in Platform Example #2 will navigate the top-level browser window directly to the destination.

Platform Example #3 Fig. G Platform Example #3 does not attempt to show a hyperlink to the user, though the short-link originally posted in the status update is visible in the browser (Fig. G). In this case the user might trust the hyperlink based on the content displayed in Platform Example #3, though navigation will proceed to <http://www.evil.com>. This is due to a redirection URI that is updated after the status update was originally posted. The spoof can be made more convincing by posting a short-link to the malicious page itself. This is because the malicious page has control over the image and text displayed in the update. Fig. G-2 The status update in Fig. G-2 was posted using a popular client application. Upon clicking the hyperlink, navigation proceeds through the bit.ly redirector to a spoof page. It is interesting to note that Platform Example #3 will pre-resolve the URI used in the status update if the update is made through the Platform Example #3 website itself. A malicious status update made in this way is a less effective spoof, as the true destination URI is revealed on hover. (See Fig. G-3)

Fig. G-3 Platform Example #4 Fig. H Platform Example #4 is a popular client application. This platform does not provide an indication of the target URI (Fig. H). The hyperlink above will navigate

the browser to a spoof page. A tooltip is also presented which the user could mistakenly trust. Fig. H-2 shows a malicious page explicitly spoofing the icon, hyperlink text, and displayed tooltip. Fig. H-2

--- page 14 ---

Platform Example #5 Fig. I Similar to Platform Example #3, Platform Example #5 displays no indication of a URI, instead relying on (Fig. I). Clicking on the hyperlink navigates the browser to <http://www.evil.com>. Fig. I-2 shows a malicious page explicitly spoofing the icon and hyperlink text displayed for a hyperlink.

--- page 15 ---

Fig. I-2 The tooltip displayed in Fig. I-2 only appears when the user hovers over the textual hyperlink. Upon clicking the hyperlink, n Originally, neither Internet Explorer nor Google Chrome displayed the destination URI as a user hovers the mouse over the large icon supplied with the status update. Per feedback, this behavior has now changed to show the destination URI on hover. Upon clicking the icon, navigation will proceed to a page under the attack redirector that performs checks for phishing, malware, and other threats.

--- page 16 ---

β^'p\_%p\$ãX\*βX4ÍbG'O?F-

--- page 17 ---

Hyperlink Spoofing Test Result Summary Number of URIs displayed to the user on view + hover  
Number of URIs displayed to the user on view\*\* Display of a trusted URI may be spoofed Attacker-controlled icon may be a hyperlink without a URI tooltip displayed on hover Attacker-controlled text may be formatted as a URI and presented to the user in a hyperlink\* **Is there an attempt to fully resolve URIs beyond short-links, facilitating better trust decisions? Twitter (circa 2011) 3 1 Yes No No Yes Twitter (circa 2012) 2 1 No No No No Platform Example #1 2\* 1\* No\* No Yes No Platform Example #2\* 1 1\* No\* No Yes Yes Platform Example #3 1\* 0\* No\* No Yes Depends on update source Platform Example #4 0\* 0\* No\* Yes Yes No Platform Example #5 1\* 0\* No\* Yes Yes No** In the table above, red represents sub-optimal behavior, pink represents marginal behavior, and green represents optimal behavior. \* Ignoring attacker-controlled text that may be formatted as a URI and presented to the user as a hyperlink. \* *This column is intended to consider touch interface scenarios where users would not normally* **With the URI presented at the beginning of the hyperlink text so as to be confusing. (Eg: without any prefix that might allow the user to understand the true destination URI)** Native Platform Example #2 status updates. Different results were observed for hyperlinks in tweets shared via Platform Example #2. It may be observed that the various platforms offer different opportunity for malicious spoofed hyperlinks, and the design of some platforms enables more robust protection from spoofing threats. McAfee McAfee is a short-On the McAfee front page the service further qualifies its intended value proposition (Fig. J).

--- page 18 ---

Fig. J Navigation to a McAfee shortened hyperlink results in a page on McAfee displaying framed content from the target site. Fig. K Note that the original URI provided to McAfee is displayed to the user in the upper frame, along with an assessment of its legitimacy (the green check mark).



recursive HTTP lookup provides servers a callback they could use to update a hyperlink destination. It might not be worth the effort to provide this functionality given the requirement for securing against malicious hyperlink updates. One final issue is that by having t.co pin hyperlinks to a final destination, t.co could conceivably fail to pin the proper destination in some cases. For example, imagine the New York Times requires authentication to access a news story, which t.co would lack. t.co might then pin all hyperlinks for New York Times news stories to a login page instead of the news story itself. Fortunately there are simple design patterns that the New York Times could follow to avoid this issue: If a request to a login page is properly authenticated, allow pass-through to the destination URI supplied via querystring. When the client navigates to the URI it will proceed to the correct destination. Special-case requests from known short-link services so they can pin the proper destination URI without authentication.

--- page 22 ---

1. On page load, perform a server-side recursive HTTP lookup for all hyperlinks. (Assumed non-starter) This solution is attractive primarily because it would preserve the ability for redirects to function as intended beyond the point at which a tweet or status update is posted. In this scheme, when a given page is loaded in the browser, hyperlinks by default would not be active. An asynchronous process in the page would update each hyperlink based on a server-side lookup of the destination URI. Given a destination URI, client-side script would then activate a given hyperlink after: a) Updating the UI for the hyperlink to show the final destination upon hover. b) Updating the actual hyperlink target to navigate the browser to the final destination upon activation. There are several drawbacks to this approach. a) Hyperlinks would not be available for a time after the page loads. b) The server must perform lookups for multiple URIs each time any web page is loaded. This would be a significantly heavier burden on the server than the current requirement for a single lookup for each tweeted URI. The scalability burden of b) would seem to be a non-starter for large platforms such as Twitter. 3) On initial status update / tweet, perform a server-side recursive lookup for all hyperlinks just as Twitter has done in the past Beyond simply using the final destination URI in UI, also use it as the hyperlink target in the tweet itself. The actual anchor text for the URI displayed in the tweet need not be replaced, though this is an option as well. sites unwilling to apply their own redirector to all hyperlinks. Mobile clients and apps could similarly follow the procedure of using the final destination URI rather than the originally provided URI as the hyperlink target. There are several drawbacks to this approach.

--- page 23 ---

- a) may be an issue to some degree due to the preemptive removal of redirects. b) Short-link services such as bit.ly could no longer provide tracking metrics. This could be resolved at the client through the use of Hyperlink Auditing. Without this support in place in the browser, short-link services would be incentivized to simply fail to redirect for the initial recursive URL lookup from the social networking platform in order to stay in the loop for any subsequent browser navigations. This would not effectively introduce any security weaknesses in the system, however it would negate the value of the feature overall. 4) A solution relying on new client-side functionality Pre-emptively validating hyperlinks at the client presents a chicken and egg problem with regard hyperacceptable to go fetch that hyperlink, thus disclosing the user had viewed it. Consequently, hyperlink validation must occur only after the user has already decided to follow



the hyperlink. In order to enable this, you might imagine that a browser could accept a specific list of acceptable domains through which any given hyperlink navigation would be restricted. The server could simply provide the data collected from its recursive HTTP request lookup. If navigation were to proceed to a domain outside this list, the browser would prompt the user for approval. cker could simply have their malicious site be one of the hyperlinks in the originally discovered redirect chain. A redirector would exist on the malicious site to facilitate this attack. Another option might be to enable an attribute on the anchor tag to specify the exact intended destination of a hyperlink. Upon navigation to an anchor, the browser would follow redirects behind-the-scenes until arriving at the intended destination. If redirection stopped prematurely or were to proceed elsewhere, the browser could then prompt the user as a warning. Otherwise it the browser would navigate directly to the destination URI. In effect this is solution #3 above, but without the performance advantage of Hyperlink Auditing.

--- page 24 ---

Too Many Hyperlinks Fig. N Revisiting the case of Twitter(Fig. N), there were actually three separate indications of the target URI present on a given tweet one in the tweet itself, one in the Twitter tooltip, and one presented by the browser. on full recursive hyperlink resolution, so the tweet URI and Twitter tooltip basically match. To help users make better trust decisions about hyperlinks, Twitter might consider removing its custom tooltip and setting the anchor URI to match the hyperlink text. This would reduce the number of indicators to two the hyperlink text and the browser indicator, each indicating the same URI. Relying solely on the browser tooltip on hover to convey a hyperlink destination URI is not advised, as there action on tablets and phones. The actual navigation on click can proceed through t.co via standard javascript hyperlink targeting techniques. Conclusion / Recommendations With the rise of social networking, users are constantly faced with the decision to click or not click on privacy. With the advent of short-links, hyperlink destinations are only getting more ambiguous and users are often left with no useful indicators to inform their decisions. To combat this threat, it is recommended that social networking sites and apps implement a solution that achieves the following goals: Allow users to make hyperlink trust decisions based on an un-ambiguous indicator of hyperlink destination. This indicator should be easily observed even on touch interface devices, which etter understand hyperlink destination URIs.

--- page 25 ---

Though it is impossible to provide an indicator that will always identify the final destination of an arbitrary hyperlink, users must be presented with hyperlink destinations that are:

- o Resolved beyond short-links such that displayed hyperlinks are unambiguous for trustworthy destinations.
- o Not capable of spoofing trustworthy destination URIs outright. (Eg: An indicated destination of microsoft.com should not navigate the user to evil.com.) Legitimate short-link services should not intentionally or unintentionally defeat mechanisms used to facilitate the determination of a final hyperlink destination. (Eg: Short-link services should avoid using framing or client-side redirects.)

Avoid presenting content (text, icons) directly under the control of the attacker as a clickable hyperlink. Or if this is unavoidable, tag the content such that the validated hyperlink destination is displayed prior to the content itself.

- o If a potentially attacker-specified icon is displayed with a status update as a hyperlink, at minimum ensure the standard browser hyperlink URI tooltip is displayed on hover. Optimally such icons should not be hyperlinks at all unless they are in some

way augmented with an indication of the destination, given touch interfaces t generally support a scenario. These goals should be possible to achieve in conformance with realistic performance, compatibility, privacy, and usability constraints. Specifically, an implementation of idea #1 or #3 as described above would suffice. Of the services tested, Platform Example #2 comes closest to defeating all identified spoofing scenarios. Acknowledgements Special thanks to Devdatta Akhawe, Manuel Caballero, Mario Heiderich, Gareth Heyes, Alex (Twitter), and Jad Boutros (Google) for contributing to this write-up and providing insightful feedback.

--- page 26 ---

þý8ZJ²ÄCR6N%GkQÎú;XC&s=gR>\*',TzM.TG8 5Rn/L5p]#;OýÝ:X<þ87Raÿð9b³;+ÿè³M+ÿð³M+ÿì³  
M+ÿì³M+ÿè³M+³⁄3/!&²S»\*50.± V?ýÔí?ýÔí/á++++ÖÄ10%#"54>32#"#"32>32@\_I~Ë◆MS-  
Ñ8jZF2OqOV◆d75c◆[MsQ4 ·/%L%%"L

--- page 27 ---

ýó%++& &"C ÅjBÿµ2®¤

--- page 28 ---

ùè¤åpp±N^@,ÍÀÀÈ!:0>1S-Í'd@SD%\$SD&ª%€ÿð@H

--- page 29 ---

,¤@+P++"C ÅjBÿµ2®¤